

Documentation technique - Data Lake Finance

Installation, architecture, pipeline, endpoints et preuves de fonctionnement

Groupe

Artemiy Smogunov - Nathan Smadja-Tubiana - Patrice Ignongui

Projet Data Lake Finance - EFREI

Lien GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

Notre objectif est de construire un data lake financier, depuis l'ingestion de données brutes jusqu'à l'analyse curated, avec orchestration Airflow, stockage MinIO, indexation Elasticsearch, base PostgreSQL, API FastAPI et contrôles de qualité.

1. Lancement du projet

Dépôt GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

Nous lançons le projet avec Docker Compose. Les services principaux exposés sont : API FastAPI sur le port 8000, Airflow sur 8080, MinIO sur 9001, PostgreSQL sur 5432, Elasticsearch sur 9200 et Redis sur 6379.

Service	URL / port	Rôle
FastAPI	http://localhost:8000/docs	Démo, ingestion, statistiques, santé
Airflow	http://localhost:8080	Orchestration du DAG financial_data_lake_pipeline
MinIO	http://localhost:9001	Stockage objet raw
Elasticsearch	http://localhost:9200	Indexation raw consultable
PostgreSQL	localhost:5432	Staging, curated et logs
Redis	localhost:6379	Cache et accélération API

Commandes de référence

```
docker compose up -d
docker compose ps
docker exec projet-data_lake_finace-api-1 python -m unittest discover -s /app/tests -v
```

2. Pipeline technique

Étape	Entrée	Sortie	Validation
Ingestion	Tickers yfinance / fichiers	Objets MinIO + docs Elasticsearch	Logs d'ingestion et statut endpoint
Staging	Docs raw par ticker	Tables nettoyées OHLCV + indicateurs	Déduplication date, types numériques, close non nul
Curated	Staging	Scores anomalie, signal, tendance	IsolationForest, seuils métier, upsert
Orchestration	Planning Airflow	DAG planifié ou manuel	Historique des runs, santé scheduler
Exposition	Base + index	API /stats, /health, /docs	Swagger et tests unitaires

Notre étape staging calcule notamment SMA20/50, EMA12/26, RSI14, MACD, bandes de Bollinger, daily_return et volatility_20. Notre étape curated réutilise ces variables, ajoute volume_zscore, applique IsolationForest puis classe les anomalies en catégories comme price_spike, flash_crash, volume_spike ou high_volatility.

3. État vérifié des services

Service	Statut	Détail
postgresql	ok	-
minio	ok	Buckets : ['raw-api-data', 'raw-financial-data']
elasticsearch	ok	8.11.0
redis	ok	-

API /health - état des services

État relevé sur notre pile Docker locale

État global

OK

Tous les services nécessaires à la chaîne data lake répondent correctement : stockage, indexation, base relationnelle et cache.

PostgreSQL

OK

-

MinIO

OK

Buckets : ['raw-api-data', 'raw-financial-data']

Elasticsearch

OK

8.11.0

Redis

OK

-

Dépôt GitHub : <https://github.com/Nathan2412/Projet-DataLake-Final>

État des services issu de la réponse réelle /health.

Nous lisons dans Airflow que le scheduler et la base de métadonnées sont en état healthy. Les champs dag_processor et triggerer peuvent être nuls dans cette configuration locale ; cela n'empêche pas le DAG de fonctionner car le scheduler et le webserver sont bien actifs.

4. Résultats techniques et justification

578 Objets raw MinIO	5972 Docs Elasticsearch	5931 Lignes staging
4923 Lignes curated	208 Anomalies	4.23% Taux anomalie

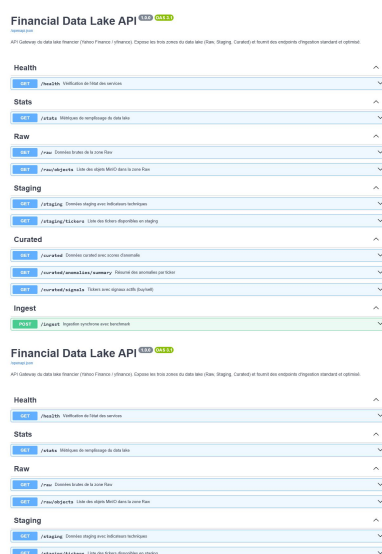
Question	Réponse technique
Pourquoi MinIO a moins de compteurs que Elasticsearch ?	MinIO compte des objets bruts. Elasticsearch compte les lignes extraites des fichiers, donc une seule archive peut produire de nombreux documents.
Pourquoi staging est inférieur au raw indexé ?	Notre staging est la première couche de qualité : doublons date/ticker, valeurs non numériques ou close manquants sont corrigés ou écartés.
Pourquoi curated est inférieur à staging ?	Nous avons 4 tickers (1 008 lignes) encore présents seulement en staging. Ce point est identifié et actionnable.
Pourquoi le taux d'anomalies est 4,24% ?	Nous ciblons 5% avec IsolationForest, mais les tickers très courts sont conservés sans apprentissage pour éviter des anomalies statistiquement faibles.

Ticker	Lignes	Anomalies	Taux	Dernière date
AAPL	869	44	5.1 %	2026-07-09
AMZN	264	14	5.3 %	2026-07-09
BRK-B	264	14	5.3 %	2026-07-09
GOOGL	264	14	5.3 %	2026-07-09
JNJ	264	14	5.3 %	2026-07-09
JPM	264	14	5.3 %	2026-07-09
META	264	14	5.3 %	2026-07-09
NVDA	264	14	5.3 %	2026-07-09

5. Performance et endpoint avancé

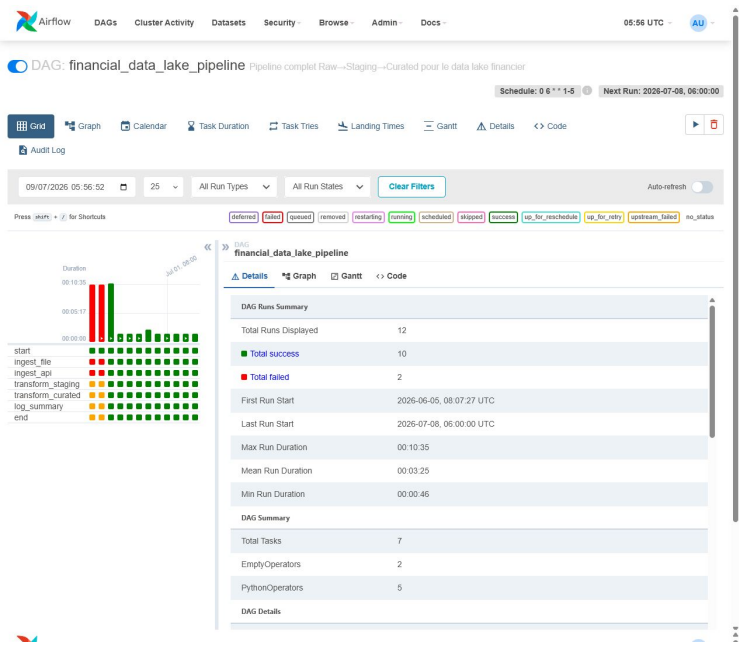
Lot	/ingest	/ingest_fast	Gain
1	4.16 s	1.86 s	55.20 %
100	70.63 s	23.61 s	66.57 %

Nous avons traité la partie avancée avec /ingest_fast et avec la couche curated. /ingest_fast combine parallélisme, upload MinIO parallèle, bulk Elasticsearch, vectorisation NumPy et insertions PostgreSQL groupées. La couche curated ajoute notre modèle non supervisé de détection d'anomalies et des signaux exploitables.

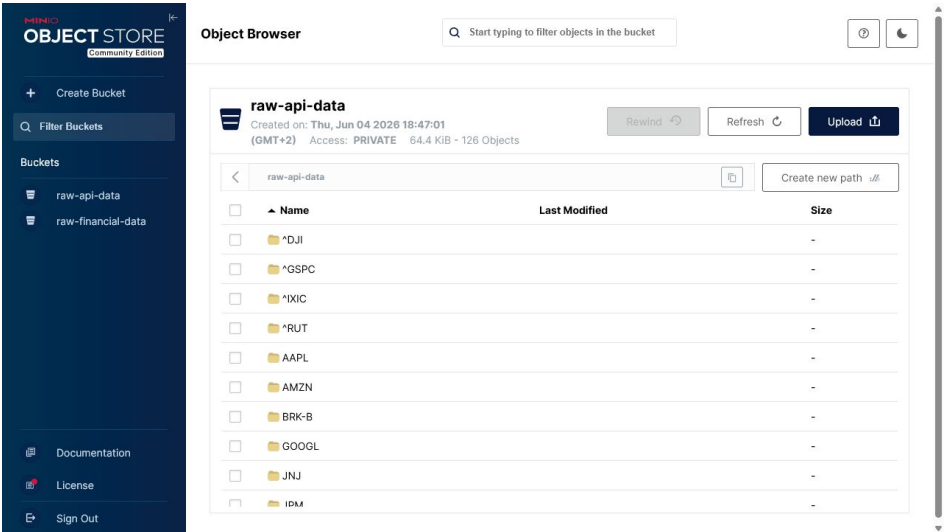


Swagger montre les endpoints disponibles pour la démonstration.

6. Captures d'exploitation



Airflow : suivi du DAG, runs et tâches récentes.



MinIO : buckets raw utilisés par le data lake.

7. Vérification et limites

- Nous avons exécuté les tests unitaires API dans le conteneur : 6 tests OK.
- Notre DAG Airflow est actif avec des derniers runs en succès ; les échecs visibles en historique datent d'essais précédents.
- Les identifiants de démonstration sont volontairement simples pour la démonstration locale : Airflow admin/admin et MinIO minioadmin/minioadmin.
- Notre limite principale est la propagation des 4 indices internationaux en curated ; elle est isolée et documentée.
- Le dépôt contient un README et les consignes afin que l'enseignant puisse relancer le projet sans dépendre de notre environnement.

Cette documentation présente nos commandes, nos ports, la logique du pipeline et les preuves de fonctionnement. Elle permet à un lecteur externe de comprendre non seulement comment lancer le projet, mais aussi pourquoi nos résultats observés sont cohérents.